MINI-PROJECT REPORT ON

“Secure Messenger Application”

SUBMITTED BY

Omkar Rasal	**MITU21BTIT0042**
Tanvi Bodke	**MITU21BTIT0061**
Ajinkya Sathe	**MITU21BTIT0005**
Prathmesh Panchmire	**MITU21BTIT0048**

UNDER THE GUIDANCE OF
Prof. Ishwari Raskar
(DEPARTMENT OF INFORMATION TECHNOLOGY)

MIT School of Engineering,
Loni Kalbhor, Pune

M.I.T. SCHOOL OF ENGINEERING
DEPARTMENT OF INFORMATION TECHNOLOGY
LONI – KALBHOR PUNE

ACKNOWLEDGMENT

The success of any project depends largely on the encouragement and guidelines of many others. This research project would not have been possiblewithout their support. We take this opportunity to express our gratitude to thepeople who have been instrumental in the successful completion of this project. First and foremost, we wish to record our sincere gratitude to the Management of this college and to our Respected Dean **Dr. Rajneesh Kaur Sachdeo**, for his constant support and encouragement in the preparation of thisreport and for the availability of library facilities needed to prepare this report. We express our sincere gratitude to our Guide, **Prof. Tushar Mane** for guiding us in investigations of this project and in carrying out experimental work. Our numerous discussions were extremely helpful. We are highly indebted to him for his guidance and constant supervision as well as for providing necessary information regarding the project & also for his support incompleting the project. We hold him in esteem for guidance, encouragement and inspiration received from him.

INDEX

Sr No.	Topic Name			Page No.
1	Abstract			01
2	List of figures			02
3	List of tables			03
4	1. Introduction			04
	1.1	Overview		04
	1.2	Motivation		05
	2. Problem Definition			06
	3. Features of Project			08
	4. Requirement Analysis			09
	5. Platform/ Technology			15
	5.1	Modules		15
	5.2	Database		17
	5.3	Diagrams		19
		5.3.1	E-R Diagram	19
		5.3.2	Flowchart	20
		5.3.3	UML Diagrams	21
	6	Output		23
	7	Code		28
5	Conclusion and Future Enhancement			55
6	Glossary			57
8	Annexure A: Annexure B:			59
9	References			60

ABSTRACT

The rapid growth of information technology, especially by using devices, linear with practical needs and convenience digital media. Today, the presence of android-based applications is quite widespread, especially in the discipline of communication media and overcoming limitations in the context of communication.

Secure messenger is a source and medium of communication that, if packaged in a practical and sophisticated way, will be able to provide solutions to the security of communication media as well as to supply impetus for user's interest and safety outcomes. This research aims to develop communication resources and communication media in the sort of a decent and useful Android-based application as an effort to increase user's interest. The research method is used for communication safety, namely users of social media who are deeply involved in the communication via online platform.

The data collection technique in this research was in the kind of documentation by journal papers through relevant references. Furthermore, to ensure that the Secure Messenger becomes more efficient, users have been provided by the some of the best benefits such as user's messages will be stored on cloud in the encrypted format. Also, user can be able to logout their account wherever necessary. Also, user will get a benefit of password hashing, which basically means that user's actual password is not going to store on the cloud instead of that password will be stored on cloud after performing the hashing method on that. Rather than providing authentication normally we will be thinking to add authentication and registration process via OTP and store all the information on the real time database(firebase). Also Online/offline status lets users know if their contacts are available or not. Unseen message count displays the number of unread messages in a conversation. Recent message shows the most recent message in a conversation. All features are useful for keeping track of conversations, knowing when it's a good time to initiate one, and quickly catching up on missed messages. Therefore, this application is proven to be able to accommodate the needs of resources and media for communication platform for various users because it can be a safe environment for communication and can be used practically without limitation of privacy.

LIST OF TABLES

Sr No.	Table Name	Page No.
1.	Database Table	19
2.	Time Estimation	30

LIST OF FIGURES

Sr No.	Figure Name	Page No.
1.	Application Logo	8
2.	Applications of Android	15
3.	Android Framework	16
4.	Firebase database icon	17
5.	ER Diagram	20
6.	Flowchart	21
7.	UML Diagram	23

1. INTRODUCTION

Secure messenger is a source and medium of communication that, if packaged in a practical and sophisticated way, will be able to provide solutions to the security of communication media as well as to supply impetus for user's interest and safety outcomes. Secure Messenger is two-way chatting application with features of high-level Encryption – Decryption. This application is secured as it uses chat Encryption-Decryption using Cryptography Algorithms. Also, this application allows users to communicate over cloud environment.

1.1 Overview



Fig (1) Application Logo

Secure Messenger is the ultimate app for messaging on Android devices that provides users with secure, private, and encrypted messages. Security and privacy are of the utmost importance, and Secure Messenger ensures that your conversations stay private and confidential. Secure Messenger uses strong encryption, ensuring that your messages and files remain secure and private, even when sent over the internet. All messages sent through the app are end-to-end encrypted, which means that only the sender and receiver can read or access their contents. Secure Messenger also has a user-friendly interface, making it easy for even beginner users to get the hang of. There is also an array of options to tailor the app to your liking, making communication more enjoyable. Overall, Secure Messenger is a great messaging app for

Android devices that provides a high level of security and privacy for all of your conversations.

1.2 Motivation

Secure Messenger is a messaging app designed to provide users with a reliable and secure chat platform. It is motivated by the rapid rise of cyber threats across the globe, including malware, phishing, ransomware, and other forms of cybercrime. Secure Messenger employs end-to-end encryption, verification of sender and recipient, and other enhanced security features to protect users' texts, videos, images, and other data from prying eyes and malicious actors. By utilizing reliable and time-tested encryption protocols, Secure Messenger is able to provide a safe chat environment even in the most hostile digital environments. The app also allows users to choose their own level of privacy, allowing them to further protect their communication with additional layers of security and control over who can access their data.

2. PROBLEM DEFINITION

In today's increasing technology there are several applications which provides a digital platform for communication but didn't provide the security for our data as well as for our personal details. Also, there is a problem related to storage consumption as many of the applications are storage consuming and it causes unnecessary flooding of files in our device which cause to lack of storage space and may have to use third party applications to storage that junk on the cloud. Many times, the device got stolen and user is unable to recover the data as well as when device got stolen, users are unable to logout their accounts from the previous devices. So, by considering all these problems we have come up with a solution named as Secure Messenger.

3. FEATURES OF PROJECT

Features of this project are -

- Authentication using Firebase
- Using Firebase Real Time Database to store Personal Details.
- Fragments to display Chats and Users.
- Cryptography Algorithms such as AES
- Auto Login/ Logout Using Firebase.
- Find friend
- Online/offline status
- Unseen message count
- Recent message

Project scope:
At the technical side, as we are developing an android application, it will beavailable for android users only.

The requirement of user is to:

- Have a great and realistic user interface.
- The application should be less complicated and easy to use.
- Application should be work properly in the emergency situations.
- User should have a way to put his experience about the application throughfeedback.
- The application should be secure.
- All the data of user should be secure.
- Application should store users' personal information in Database.
- Messages Should be end to end encrypted.

4. REQUIREMENT ANALYSIS

Requirement Analysis is the first phase of software development process. This phase focuses tounderstand the problem. Requirement Analysis is on identifying what is need from these systems, not how the system will achieve its goals. In this phase often at least two parties are involved in Software Development-a client and a developer. The developer has to develop the system to satisfy the clients 'needs. The developer and client arrange a meeting and discuss his/her own views.

Requirement gathering for the Secure Messenger app started with brain storming and discussion with other students as to what features are the most essential in communication. This led to the most basicand initial draft of requirements for the application. A brief study of the functionality of the devices helped us to refine and narrow down the requirements even further. One important thing to learn for these devices was the simplicity of their design. This helped us to design an effective and simple UI design for my application. The next step for requirements understanding was to look for existing solutions and similar applications in the Android market. A careful study of these applications, adding other important features and removing unnecessary features was done.

In the software requirement we are dealing with the requirements of the proposed system, that'sthe capabilities of that system, which is yet to be developed, should have. The softwarerequirement specification (SRS) is a document that completely describes what the proposed software should do without describing how the software will do it. So, the basic goal of Requirement Phase is to produce the SRS, which describes the complete external behavior of theproposed software. The basic aim of problem analysis is to obtain the clear understanding of the needs of the clients and the user, what exactly described from the software, and what the constraints on the solution are? This involves a meeting of user and developers.

4.1 Hardware Requirements

The Collection of internal electronic circuits and external physical devices used in building a computer is called Hardware.
The minimum hardware requirement specification for developing this project is as follows:

- Processor: Intel core i5 CPU 2.40 GHz
- RAM: 3GB-8GB RAM
- Hard Disk: 2GB
- Monitor: Standard Color Monitor
- Keyboard: Standard Keyboard
- Mouse: Standard Mouse
- Device: Desktop Computer
- Screen Resolution: 1280*800
- Android Smartphone

4.2 Software Requirements

A set of programs associated with the operation of a computer is called software. Software is thepart of the computer system which enables the user to interact with several physical hardware devices.
The minimum software requirement specifications for developing this project are as follows:

- Operating System: Windows 2007,2008,2010, Ubuntu
- Documentation Tool: MS Office
- Database: Firebase
- Software: Android Studio

4.2.1 Front-End

As our implementation is in android, as front end, what we have used is XML (Extensible MarkupLanguage) and the core programming language that we are using is java.



XML is a markup language much like HTML used to describe data. XML tags are not predefined in XML so we have defined our own Tags. XML as itself is well readable both by human and machine. Also, it is scalable and simple to develop. In Android we use xml for designing our layouts because xml is lightweight language so it doesn‘t make our layout heavy.

4.2.2 Back-End

The core language that we have used as backend is java because of its-

- Simplicity
- Multi-Threading feature
- Security
- Open-source libraries
- Rich Ecosystem
- Robustness & scalability



4.3 Android:



Fig (2) Applications of Android

- Smooth work, fast operating speed, and flawless performance. The native app is builtfor a specific OS and take full advantage of the processing speed of the device.
- Great design and user experience possibilities.
- For instance, the implementation of sophisticated UX/UI design and animation Nativeapp UI allows users to understand navigation within the app.
- Good integration with the hardware of the device.
- Easy to launch in the App Store.

- When it comes to the question of native vs hybrid app or a mobile web app, nativeapplications follow the OS (Android or iOS) guidelines completely.
- Better security & reliability (apps developed natively are checked and approved by theApp Store).
- Easy to implement new sophisticated features that the market demands (AI,VR,AR,IoT

  ,etc.) in native vs hybrid app.
- Ability to operate offline. Internet connection is necessary if there is correspondingfunctionality required for native app development SDK for developers.
- It provides half-ready, regularly updated solutions and libraries that interact properlywith a given operating system.

Framework:

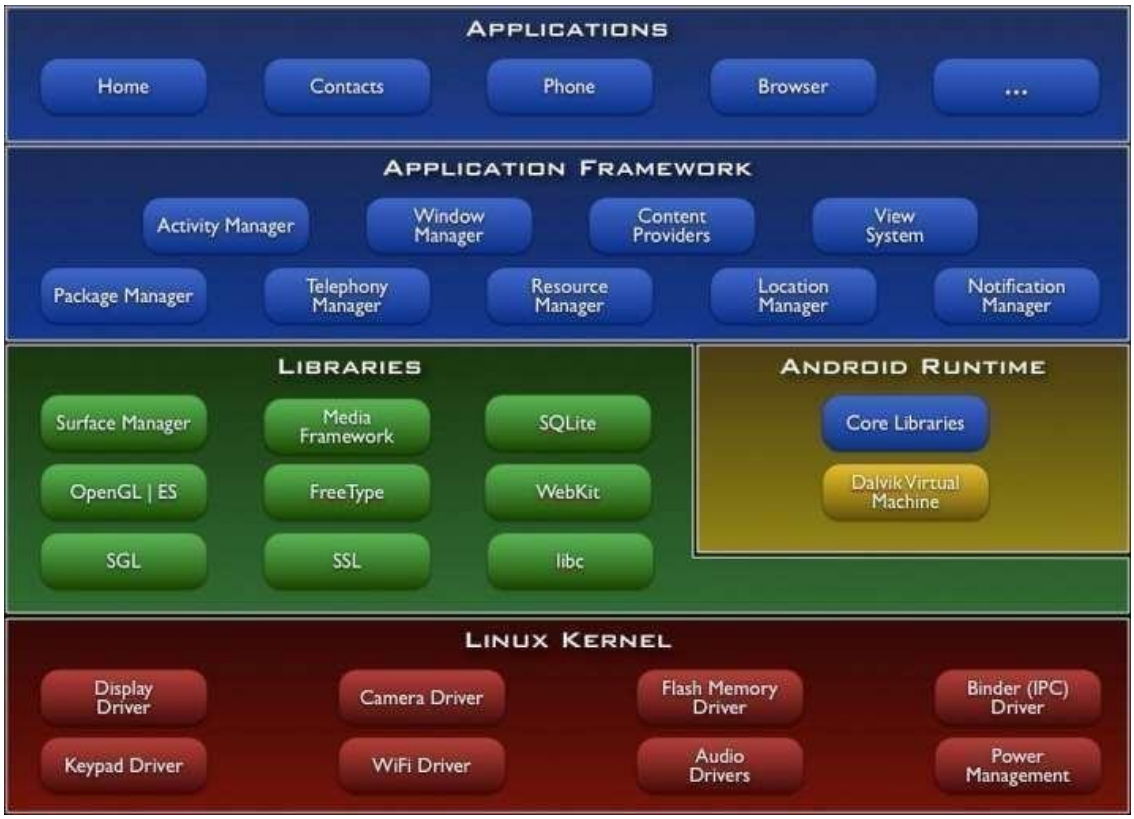


Fig (3) Android Framework

4.3 FIREBASE



Fig (4) Firebase icon

Firebase is a cloud-hosted NoSQL database service provided by Google, offering real-time data synchronization across multiple users and devices. Firebase stores data as JSON objects, enabling real-time synchronization, where changes to data made by one user are immediately propagated to all connected users. It supports live data updates, automatic data scaling and partitioning, on-the-fly indexing, and simple queries. Firebase is designed to help developers build real-time applications quickly and easily. With Firebase, you can use the same database engine and APIs to develop applications for web, iOS and Android.

Firebase is designed to give developers an easy way to build applications that use real-time data, with multi-user collaboration. Firebase provides all the necessary tools to securely store and synchronize data, and supports several development languages. It also offers a suite of services to help manage and track user engagement with an application, including analytics, remote config, and A/B testing. Firebase also integrates with third-party services including Google Cloud Storage and BigQuery, allowing you to access up to 20GB of data storage.

Firebase is also optimized for mobile devices and provides a range of features specifically for Android and iOS. Firebase's SDK for iOS and Android gives developers access to an extensive range of features and tools, such as push notifications and powerful database synchronization APIs. You can also sync data with cloud storage and analytics, allowing you to build complex applications with a single codebase. With Firebase, developers can also track user activities and engagement with their apps, as well as use A/B testing and run remote config tasks. In addition, Firebase supports authentication and enables developers to easily store user data.

5. PLATFORM/ TECHNOLOGY

5.1 Flowchart

A diagram is a simplified graphic design showing the appearance, structure, or workings of theproject or software; a schematic representation.

Here we are including:

5.3.1 Entity Relationship diagram

5.3.2 Data flow diagram

5.3.3 UML Diagrams

5.3.1 ER Diagram:

- An Entity Relationship (ER) Diagram is a type of flowchart that illustrates how –entities such as people, objects or concepts relate to each other within a system.
- ER Diagrams are most often used to design or debug relational databases in the fields of software engineering, business information systems, education and research.
- Also known as ERDs or ER Models, they use a defined set of symbols such as rectangles, diamonds, ovals and connecting lines to depict the interconnectedness of entities, relationships and their attributes.
- They mirror grammatical structure, with entities as nouns and relationships as verbs.

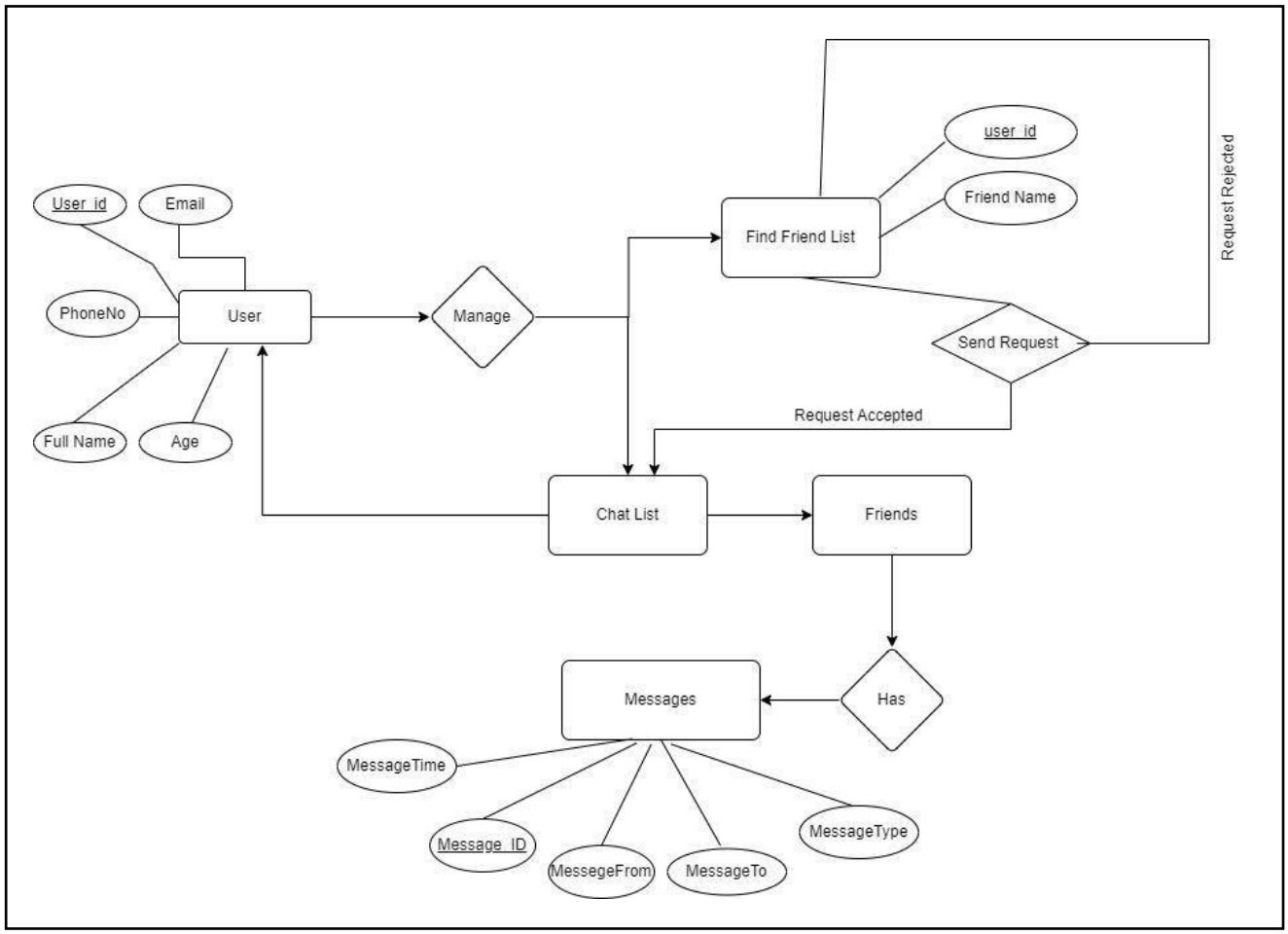


Fig (5): ER Diagram

5.3.2 Flow Chart:

A flowchart is a pictorial representation of a process, which can help people to better understand the steps and decisions that need to be made in a given situation. It is often used to document and analyze processes or to map out a system, such as a computer program or workflow. Flowcharts are composed of symbols, such as circles and arrows, which represent steps in the process, while other symbols may be used to represent decision points and actions.

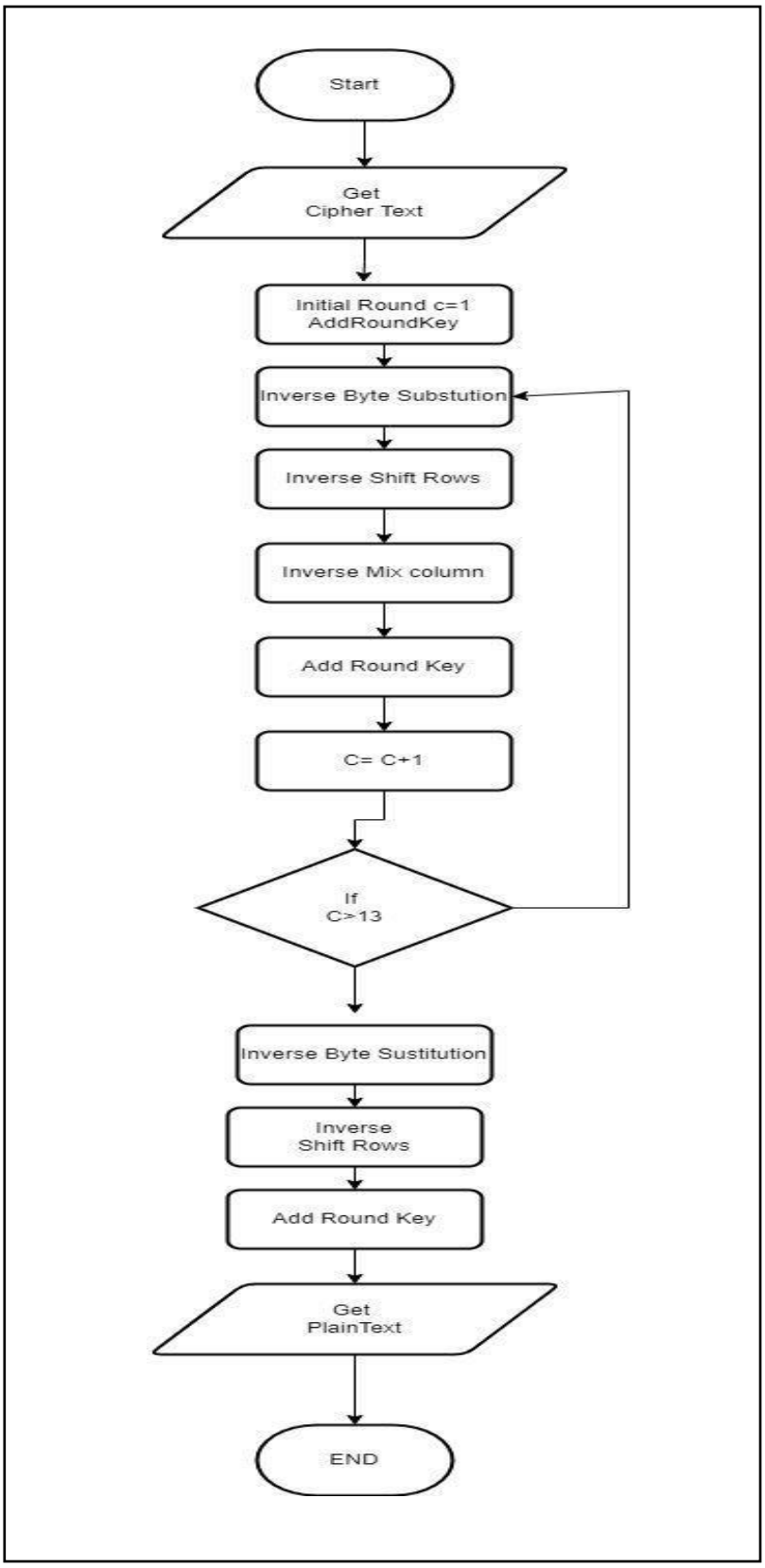


Fig (6) Flowchart of encryption for AES-256 algorithm

5.3.3 UML Diagram

A UML diagram shows the unified visual presentation of the UML (Unified Modeling Language) system intending to let developers or business owners understand, analyze, and undertake the structure and behaviors of their system.

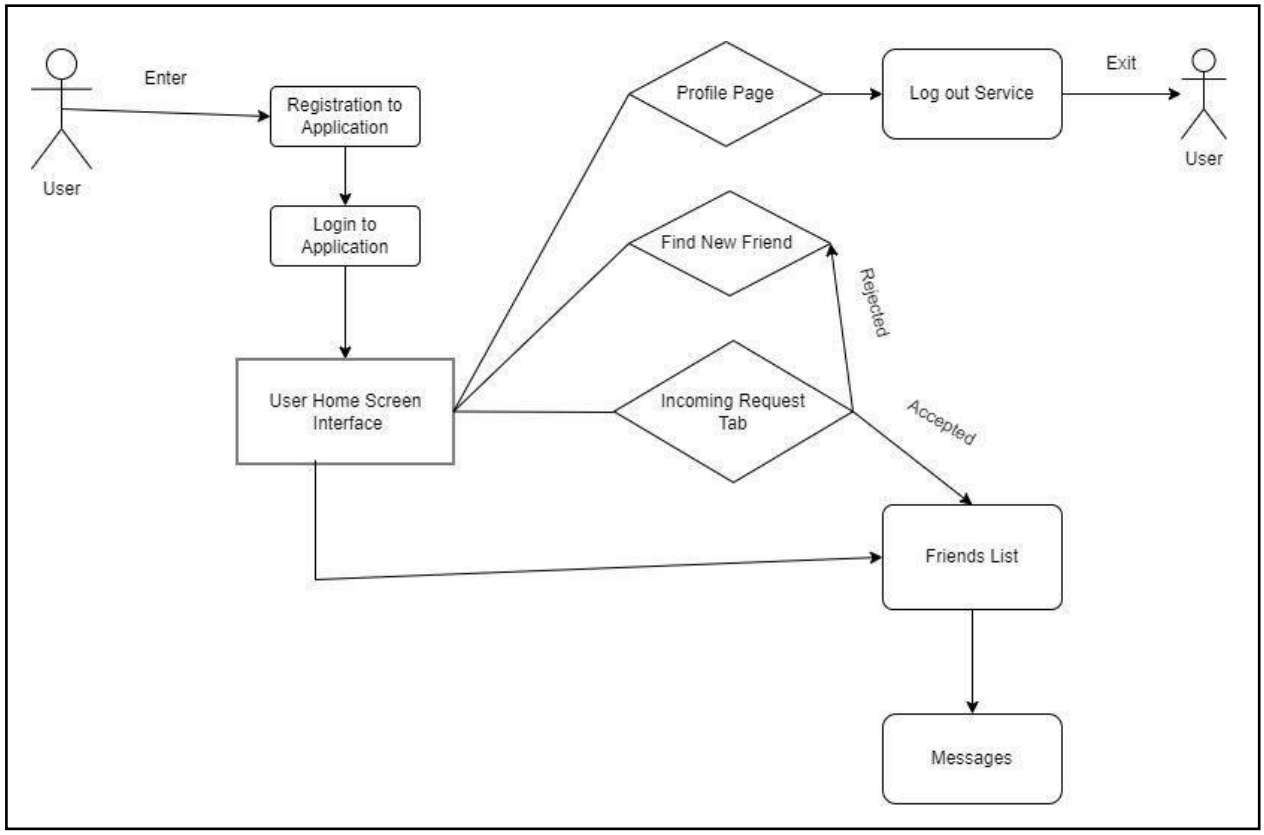


Fig (7): UML Diagram

Why Use UML Diagrams?

UML diagramming has many benefits for both software developers and businessman, and the most key advantages are:

3.3.2.1 Problem-Solving - Enterprises can improve their product quality and reduce cost especially for complex systems in large scale. Some other real-life problems including physical distribution or security can be solved.

3.3.2.2 Improve Productivity - By using the UML diagram, everyone in the team is on the same page and lots of time are saved down the line.

3.3.2.3 Easy to Understand - Since different roles are interested in different aspects of the system, the UML diagram offers non-professional developers, for example,

stakeholders, designers, or business researchers, a clear and expressive presentation of requirements, functions and processes of their system.

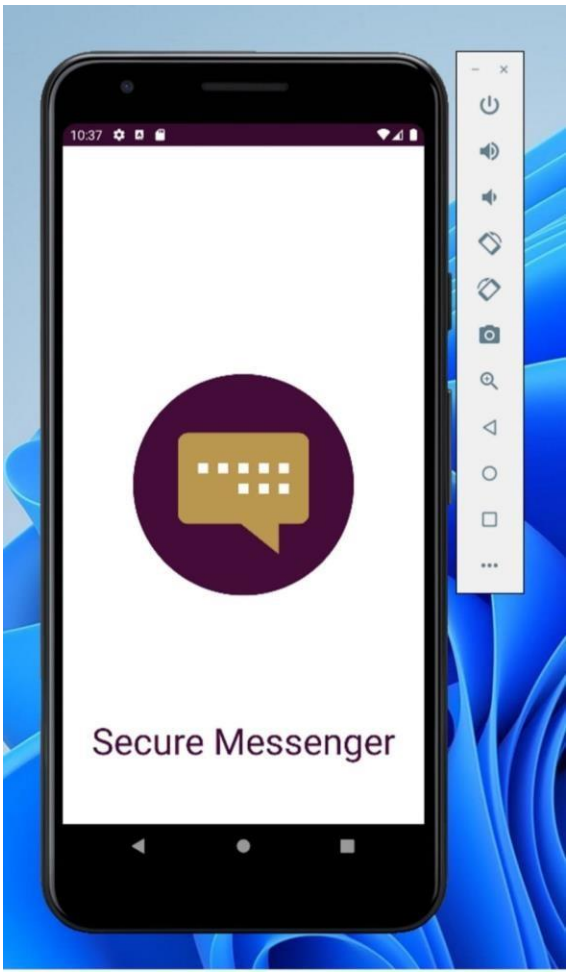
6. OUTPUT:

6.1 Modules

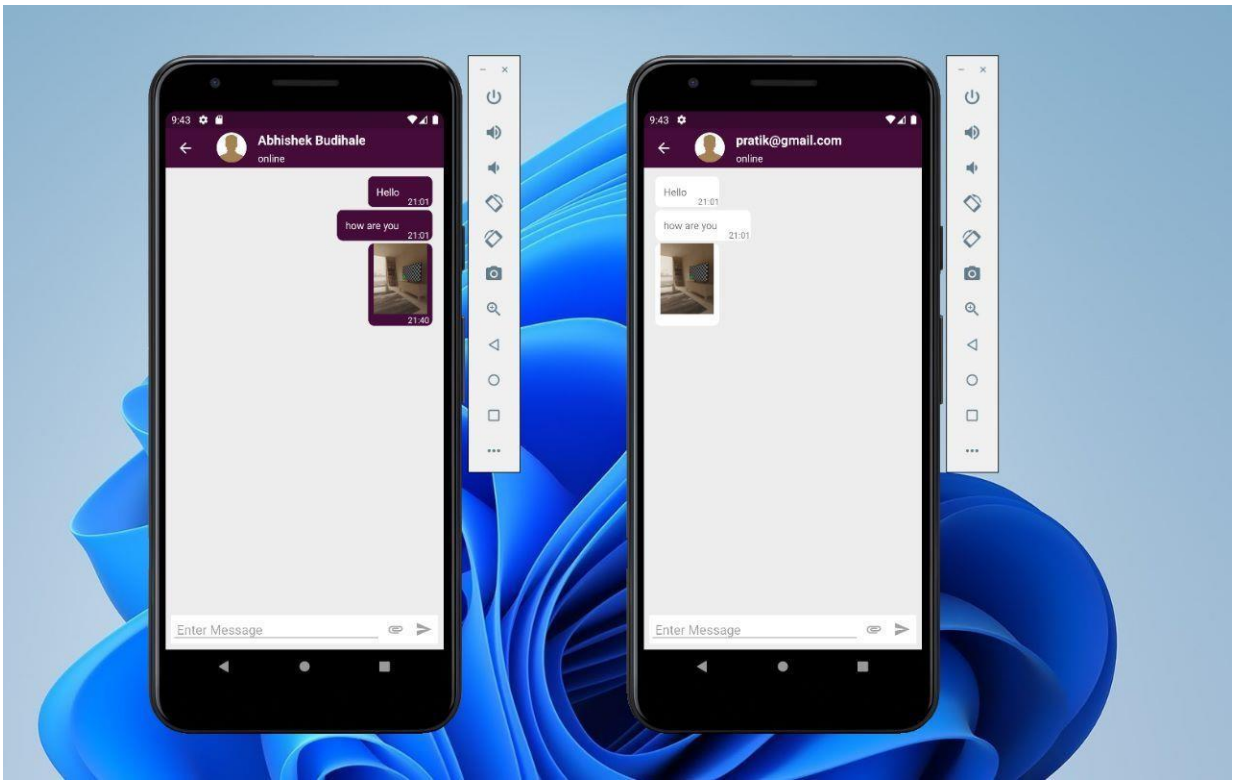
Modular design is a design approach that creates things out of independent parts with standard interfaces. This allows designs to be customized, upgraded, repaired and for parts to be reused.

Modules provide a container for your app's source code, resource files, and app level settings, such as the module-level build file and Android manifest file. Each module canbe independently built, tested, and debugged.

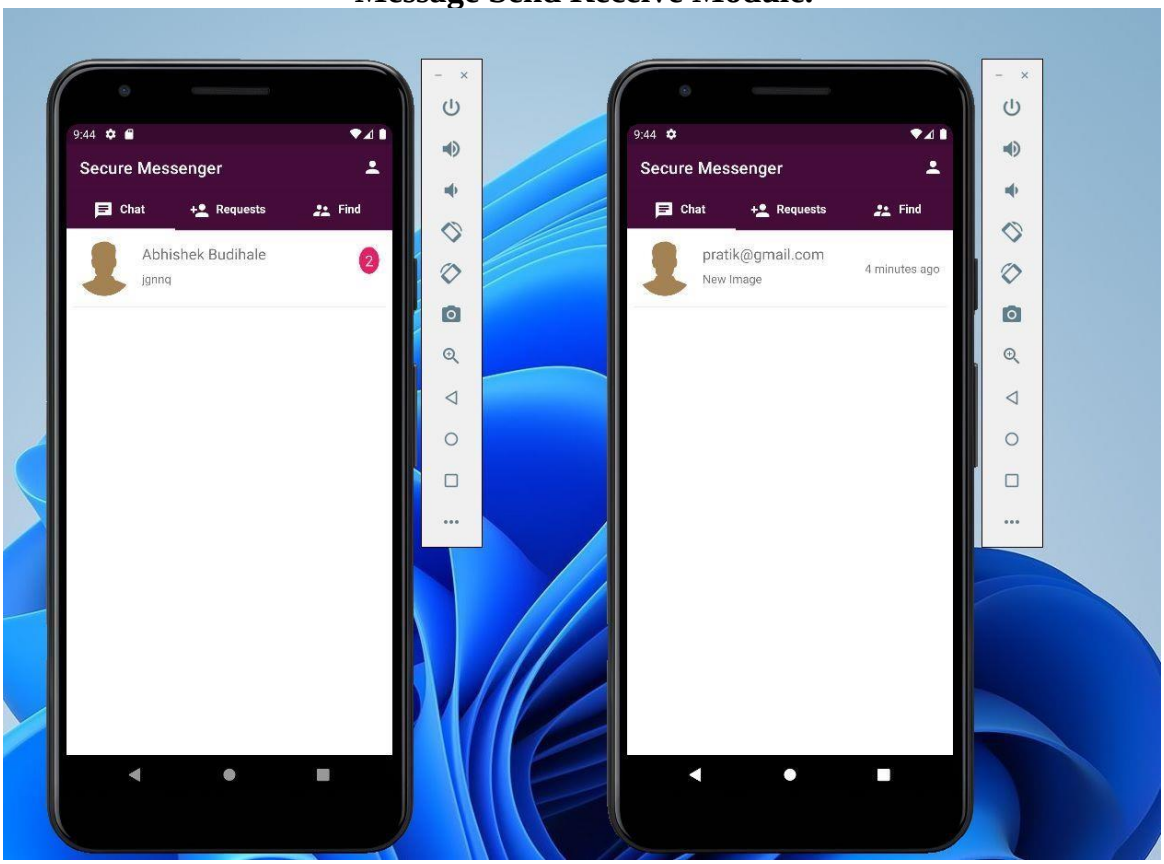
Modules Used in Secure Messenger:



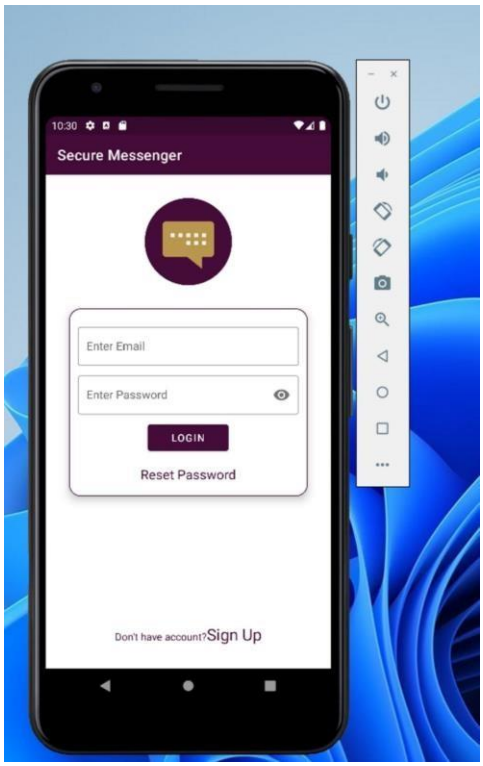
SPLASH SCREEN

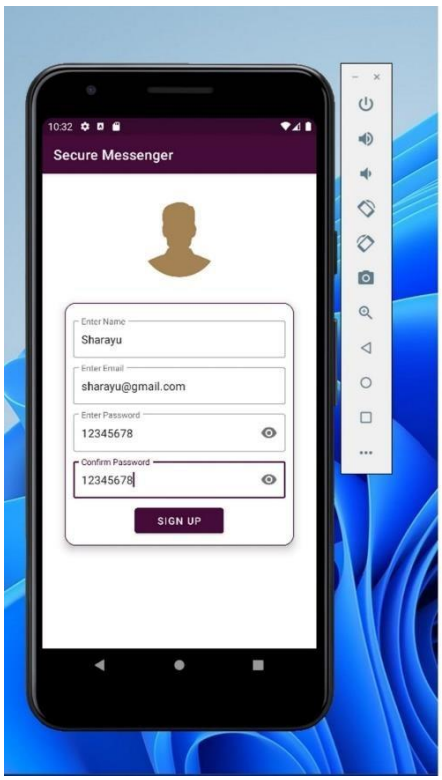


Message Send Receive Module.

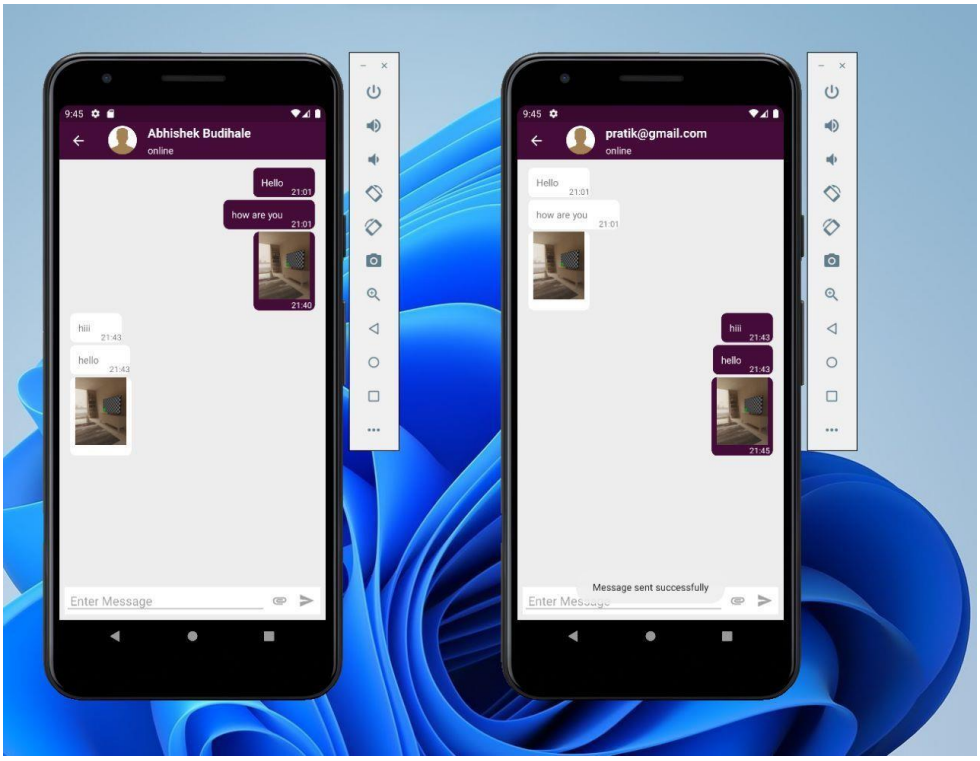


Unseen Message Count

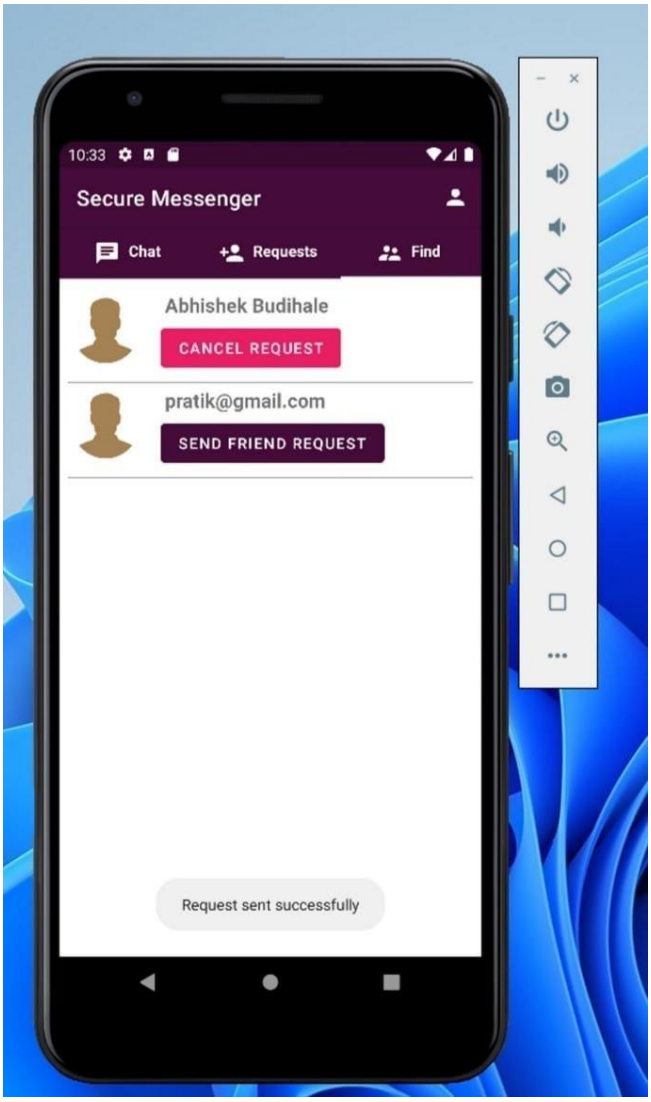


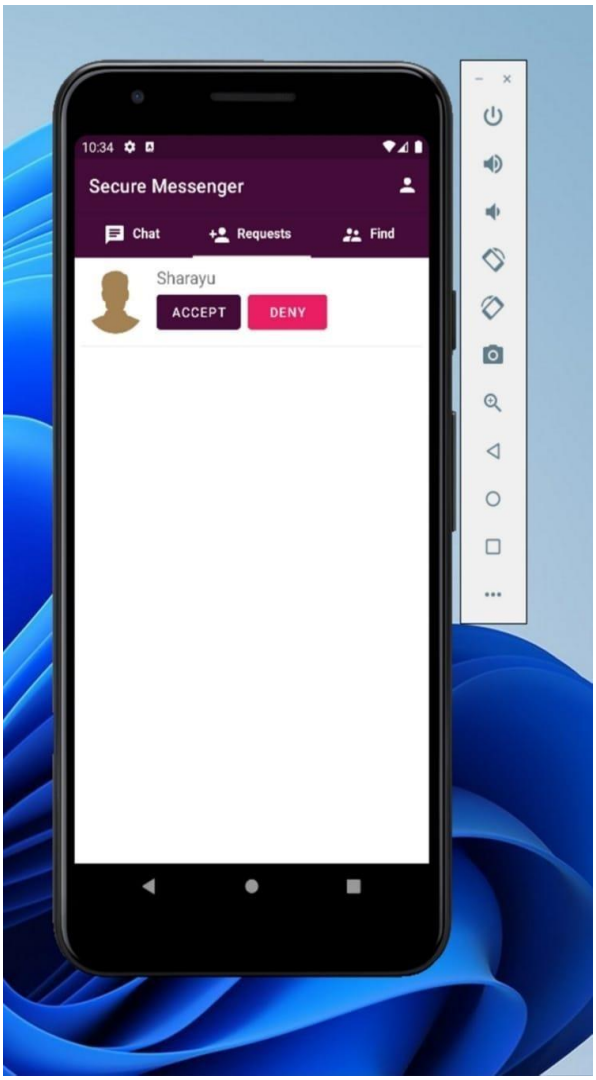


Login and registration module



Conversation Screen





Friend Request Sent/Receive Module.

6.2 Database

A database is an organized collection of data, so that it can be easily accessed and managed. You can organize data into tables, rows, columns, and index it to make it easier to find relevant information. A database is a systematic collection of data. They support electronic storage and manipulation of data. Databases make data management easy.

The database management system (DBMS) is the software that interacts with end users, applications, and the database itself to capture and analyze the data. The DBMS software additionally encompasses the core facilities provided to administer the database. The sum total of the database, the DBMS and the associated applications can be referred to as a "database system". Often the term "database" is also used to loosely refer to any of the DBMS, the databasesystem or an application associated with the database.

- We have used **Firebase** as a database in our project.

How does Firebase work?

Firebase is a serverless application development platform from Google. It is a Backend-as-a-Service (BaaS) offering that provides developers a way to integrate a variety of services into their projects, such as databases, user authentication, file storage and hosting, machine learning and analytics. Firebase is a relatively new product, but it has quickly become a popular choice among developers due to its wide range of features, scalability, and pricing.

Firebase is built on top of Google Cloud Platform, which provides a secure foundation for running applications. It is composed of a suite of different products, such as Firebase Cloud Storage, Firebase Authentication and Firebase Hosting, all of which offer users various ways to interact with the cloud. Firebase Storage is used to store user data and files, while Firebase Authentication provides secure user authentication and authorization. Firebase Analytics tracks user behavior and helps developers gain insights into how their app is being used

Database Tables:

1. Message Sent Table:

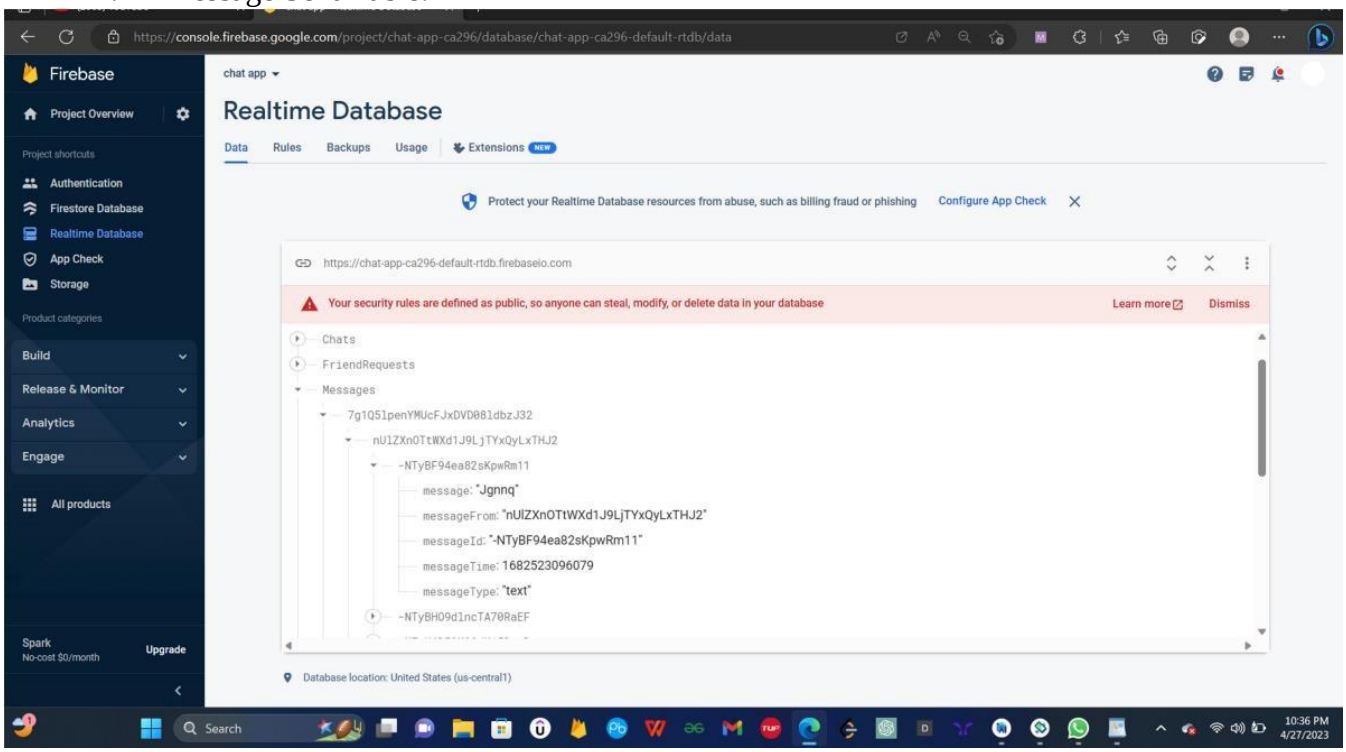
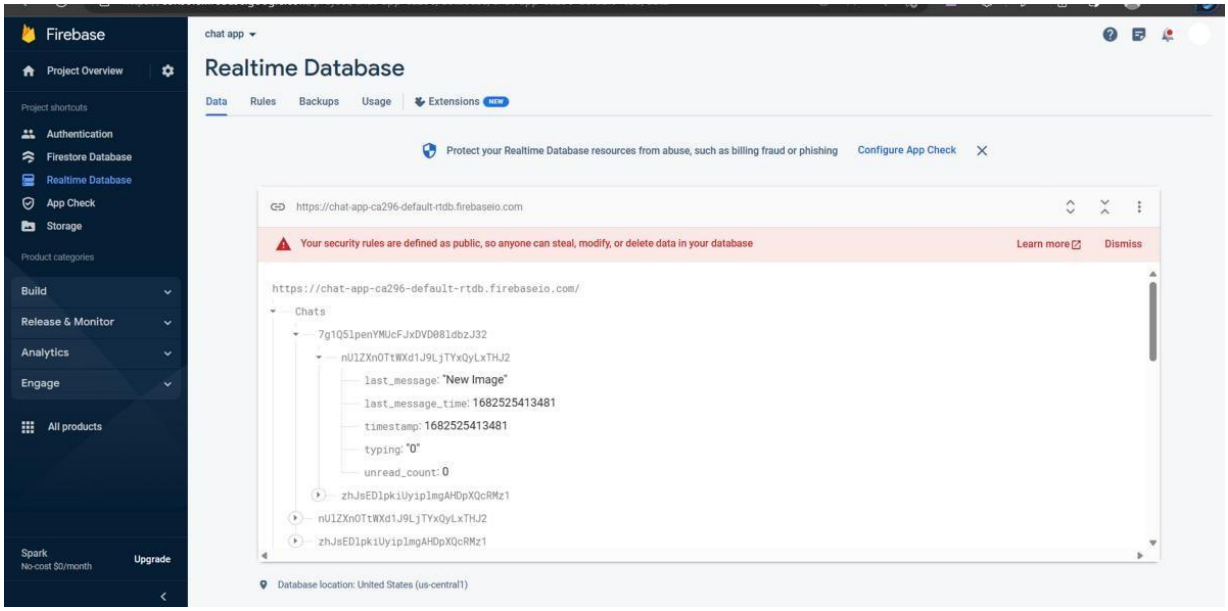


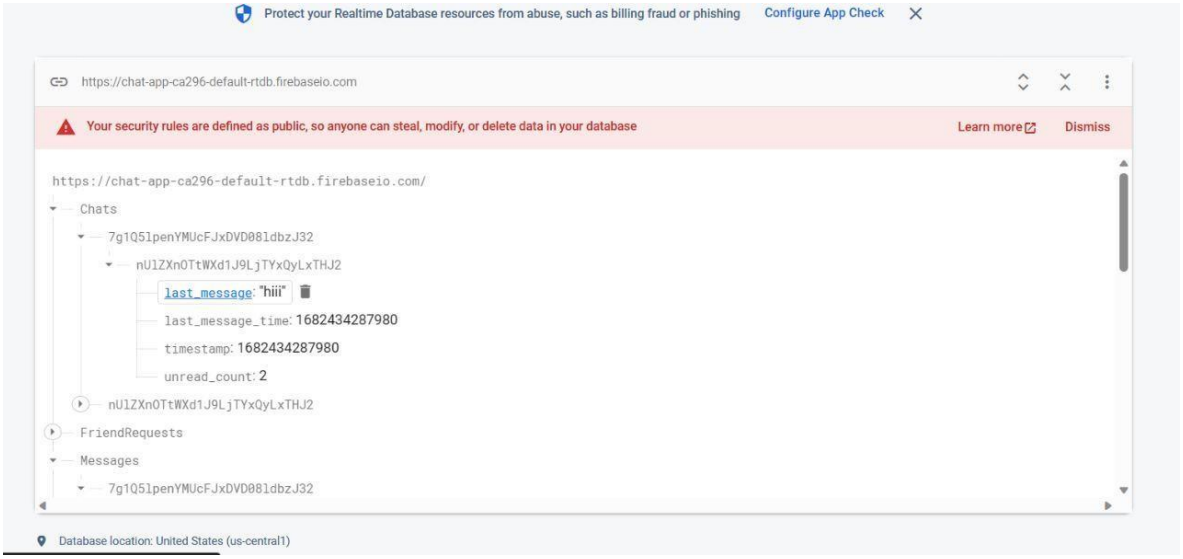
Image Sent Table :



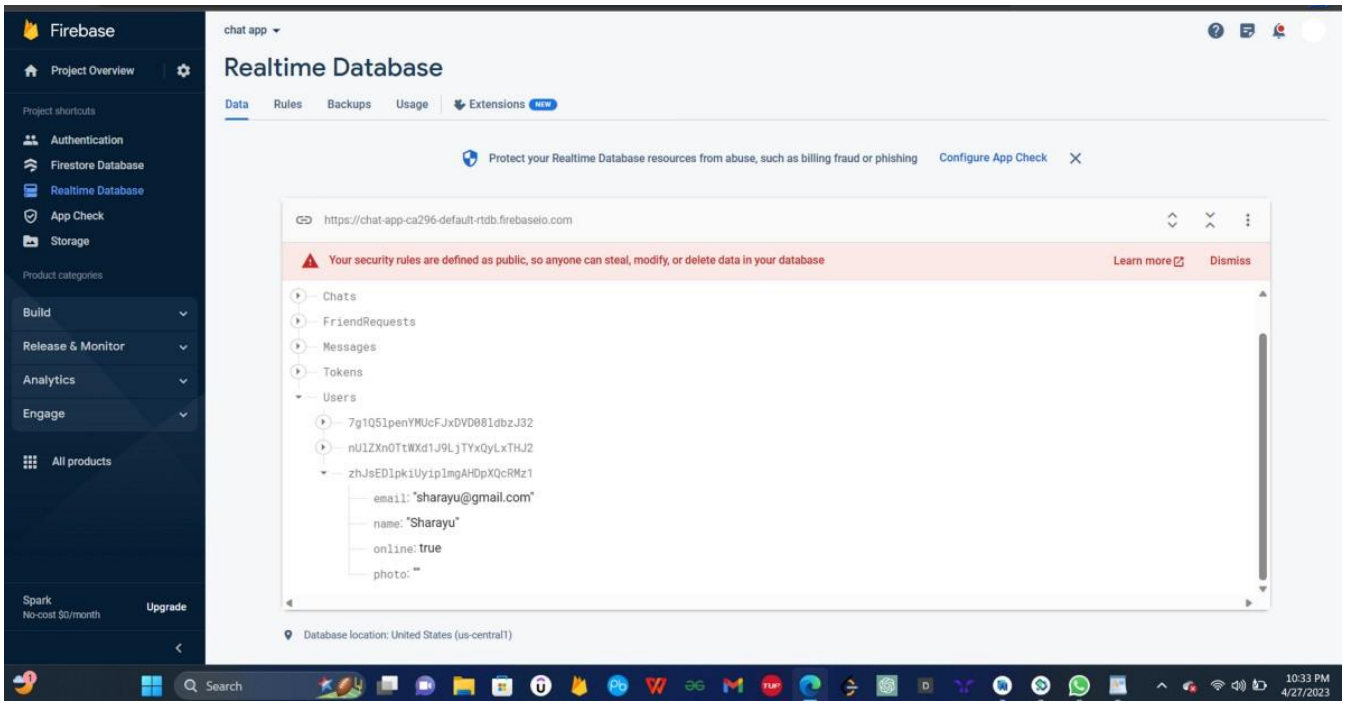
2. Online Offline Table :



3. Unseen Message Count Table :



4. Authentication Table:



CODE

1) ChatActivity.java

```
package com.example.section7findfriend.chat;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.appcompat.app.ActionBar;
import  androidx.appcompat.app.AppCompatActivity;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;
import androidx.swiperefreshlayout.widget.SwipeRefreshLayout;

import android.annotation.SuppressLint;
import android.os.Bundle;
import android.util.Log;
import android.view.MenuItem;
import android.view.View;
import android.view.ViewGroup;
import android.widget.EditText;
import android.widget.ImageView;
import android.widget.LinearLayout;
import android.widget.TextView;
import android.widget.Toast;

import com.example.section7findfriend.EncryptionAlgo.Algo;
import com.example.section7findfriend.R;
import com.example.section7findfriend.data.Constant;
```

```
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.database.ChildEventListener;
import com.google.firebase.database.DataSnapshot;
import com.google.firebase.database.DatabaseError;
import com.google.firebase.database.DatabaseReference;
import com.google.firebase.database.FirebaseDatabase;
import com.google.firebase.database.Query;
import com.google.firebase.database.ServerValue;
import com.google.firebase.database.ValueEventListener;
import com.google.firebase.storage.internal.Util;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;

//we have implemented View OnCLickListerner because we want to listen which of above click listerner called

public class ChatActivity extends AppCompatActivity implements View.OnClickListener {


    private ImageView ivSend;
    private EditText etMessage;

    //.....
    DatabaseReference mRootRef;
    FirebaseAuth firebaseAuth;
    private String currentUserId,chatUserId,chatUserName;
    //.....
```

```
//Chating Layout

RecyclerView rvMessage;
SwipeRefreshLayout srlMessage;
MessagesAdaptor messagesAdaptor;
List<MessageModel> messageModelList;
List<MessageModel> messageModelList2;

//Custom Action Bar.....
TextView tvUserName;
ImageView ivProfile;
LinearLayout ll_cab;
//.....


//.....
//we want to show only 30 message on first screen then

int currentPage = 1;
static final int RECOORD_PER_PAGE = 30;

DatabaseReference databaseReferenceMessage;
ChildEventListener childEventListener;
ChildEventListener childEventListener2;

//.....
```

```
    @SuppressLint("MissingInflatedId")
    @Override
    protected void onCreate(Bundle savedInstanceState) {




//*************************************************************************
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_chat);
//_____________________________________________________________________
     //Custom Action Bar


        ActionBar actionBar =getSupportActionBar();
        if(actionBar!=null){
            actionBar.setTitle("");
            ViewGroup actionBarLayout = (ViewGroup)
getLayoutInflater().inflate(R.layout.custom_action_bar,null);

    //       actionBar.setDefaultDisplayHomeAsUpEnabled(true);
            actionBar.setHomeButtonEnabled(true);
            actionBar.setElevation(0);


            actionBar.setCustomView(actionBarLayout);

actionBar.setDisplayOptions(actionBar.getDisplayOptions()|ActionBar.DISPLAY_SHOW_CUSTOM);
```

```
    }

    tvUserName=findViewById(R.id.tv_cab);
    ll_cab=findViewById(R.id.ll_profile);
//-------------------------------------------------------------------
    //Binding Components
    ivSend=findViewById(R.id.ivSend);
    etMessage=findViewById(R.id.etMessage);
    rvMessage=findViewById(R.id.rvMessages);
    srlMessage=findViewById(R.id.srlMessages);

    //Called on View Click Action
    ivSend.setOnClickListener(this);

    firebaseAuth= FirebaseAuth.getInstance();  //FirebaseAuth
    mRootRef=FirebaseDatabase.getInstance().getReference();//Ref to Database root
    currentUserId=firebaseAuth.getCurrentUser().getUid();

    //check to get chatUserId from chat Activity ....
    //Extra information can put in intent thorough Put Extra....
    if(getIntent().hasExtra("user_key"))
    {
       chatUserId = getIntent().getStringExtra("user_key");
    }
    if(getIntent().hasExtra("user_Name")){
       chatUserName=getIntent().getStringExtra("user_Name");
    }
    tvUserName.setText(chatUserName);
```

```
//set tvUserName as chatUserName



Log.d("","User_key : After "+chatUserId);



messageModelList = new ArrayList<>(); //intialize ModelList
messageModelList2 = new ArrayList<>();

messagesAdaptor = new MessagesAdaptor(this,messageModelList); //Create MessageAdaptor

rvMessage.setLayoutManager(new LinearLayoutManager(this));  //Need to set Layout

rvMessage.setAdapter(messagesAdaptor);  //set Adaptor on Recycle VIew

Log.d("","Before Load Messages");
LoadMessages();
Log.d("","After Load Messages");
//.....
rvMessage.scrollToPosition(messageModelList.size()-1);
srlMessage.setOnRefreshListener(new SwipeRefreshLayout.OnRefreshListener() {
    @Override
    public void onRefresh() {
        currentPage++;
        LoadMessages();
    }
});
```

```
//*************************************************************************************
*******
    }


    private  void sendMesssage(String msg,String msgType, String pushId)
    {
        try{

            if(!msg.equals(""))
            {
                //HashMap created for storing Data

                HashMap messageMap = new HashMap();
                messageMap.put("messageId",pushId);
                messageMap.put("message",msg);
                messageMap.put("messageType",msgType);
                messageMap.put("messageFrom",currentUserId);
                messageMap.put("messageTime", ServerValue.TIMESTAMP);


                String currentUserRef = "messages"+"/"+currentUserId+"/"+chatUserId;

                String chatUserRef = "messages"+"/"+chatUserId+"/"+currentUserId;
```

```
            //.....

            HashMap messageUserMap = new HashMap();

            //Structure of Storing message in firebase Datbase
            messageUserMap.put(currentUserRef+"/"+pushId,messageMap);
            messageUserMap.put(chatUserRef+"/"+pushId,messageMap);

            //after sending message message box should be empty.....
            etMessage.setText("");

//*************************************************************************************
*****************************

        //TO Update root childern Ref .....

            mRootRef.updateChildren(messageUserMap, new DatabaseReference.CompletionListener() {
                @Override
                public void onComplete(@Nullable DatabaseError error, @NonNull DatabaseReference
ref) {

                    if(error!=null)
                    {
                        Toast.makeText(getApplicationContext(),getString(R.string.failed_to_send_message,
error.getMessage())
                        ,Toast.LENGTH_LONG).show();
                    }
                    {
```

```
                    //
Toast.makeText(getApplicationContext(),getString(R.string.message_send_sucessfully),Toast.LENGTH
_LONG).show();
                }
            }
        });

//**********************************************************************************************
**********

        }

    }
    catch (Exception ex)
    {
        Toast.makeText(getApplicationContext(),getString(R.string.failed_to_send_message,
ex.getMessage())
            ,Toast.LENGTH_LONG).show();
    }
}

private void  LoadMessages()
{
    Log.d("","Inside Load Messages");
    messageModelList.clear();
    databaseReferenceMessage=mRootRef.child("messages").child(currentUserId).child(chatUserId);
```

```
        Query query = databaseReferenceMessage.limitToLast(currentPage * RECOORD_PER_PAGE);
//.....
//.....
        if(childEventListener!=null)
            query.removeEventListener(childEventListener);


        childEventListener = new ChildEventListener() {
            @Override
            public void onChildAdded(@NonNull DataSnapshot snapshot, @Nullable String
previousChildName) {


                MessageModel message1 = snapshot.getValue(MessageModel.class);
                Log.d("","Before Size"+messageModelList.size());
                messageModelList.add(message1);
                Log.d("","Size"+messageModelList.size());
                messagesAdaptor.notifyDataSetChanged();
                //we want to show latest Message.....
                rvMessage.scrollToPosition(messageModelList.size()-1);
                srlMessage.setRefreshing(false);



            }
            @Override
            public void onChildChanged(@NonNull DataSnapshot snapshot, @Nullable String
previousChildName) {


            }
```

```
        @Override
        public void onChildRemoved(@NonNull DataSnapshot snapshot) {


        }


        @Override
        public void onChildMoved(@NonNull DataSnapshot snapshot, @Nullable String
previousChildName) {


        }


        @Override
        public void onCancelled(@NonNull DatabaseError error) {
            srlMessage.setRefreshing(false);


        }
      };
//-----------------------------------------------------------------------------------------------
      query.addChildEventListener(childEventListener);
      Log.d("","Inside After Load Messages");
   }


//*************************************************************************************
*******


   //Onlick Method of View OnCLick Class
   @Override
   public void onClick(View view) {
```

```
        //TO check which of the Following Onclick pressed.....
        switch (view.getId()){
            case R.id.ivSend:
                DatabaseReference userMessagePush
=mRootRef.child("messages").child(currentUserId).child(chatUserId).push();
                //Push() : will generate Unique messageId
                String pushId = userMessagePush.getKey();
                String encText;
               //  encText= Algo.encrypt(String.valueOf(etMessage.getText()));
                String text= String.valueOf(etMessage.getText());
               // encText=Algo.SimpleEncryption(text);
                encText=Algo.encrypt(text);
                sendMesssage(encText,"text",pushId);
                break;
        }

    }

    @Override
    public void onPointerCaptureChanged(boolean hasCapture) {
        super.onPointerCaptureChanged(hasCapture);
    }

    @Override
    public boolean onOptionsItemSelected(@NonNull MenuItem item) {
        int itemId= item.getItemId();
        switch (itemId)
        {
            case android.R.id.home:
```

```
                finish();
                break;
            default:
                break;
        }
        return super.onOptionsItemSelected(item);
    }
}
```

2) Algo.java

```
package com.example.section7findfriend.EncryptionAlgo;

import android.os.Build;

import java.security.InvalidAlgorithmParameterException;
import java.security.InvalidKeyException;
import java.security.NoSuchAlgorithmException;
import java.security.spec.InvalidKeySpecException;

import javax.crypto.BadPaddingException;
import javax.crypto.IllegalBlockSizeException;
import javax.crypto.NoSuchPaddingException;


import javax.crypto.Cipher;
import javax.crypto.SecretKey;
import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.IvParameterSpec;
import javax.crypto.spec.PBEKeySpec;
```

```
import javax.crypto.spec.SecretKeySpec;
import java.nio.charset.StandardCharsets;
import java.security.InvalidAlgorithmParameterException;
import java.security.InvalidKeyException;
import java.security.NoSuchAlgorithmException;
import java.security.spec.InvalidKeySpecException;
import java.security.spec.KeySpec;
import java.util.Base64;
import javax.crypto.BadPaddingException;
import javax.crypto.IllegalBlockSizeException;
import javax.crypto.NoSuchPaddingException;

public class Algo {

    private static final String SECRET_KEY = "123456789";
    private static final String SALTVALUE = "abcdefg";



    public static String encrypt(String strToEncrypt)
    {
        try
        {
            /* Declare a byte array. */
            byte[] iv = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
            IvParameterSpec ivspec = new IvParameterSpec(iv);
            /* Create factory for secret keys. */
            SecretKeyFactory factory = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256");
            /* PBEKeySpec class implements KeySpec interface. */
```

```
        KeySpec spec = new PBEKeySpec(SECRET_KEY.toCharArray(), SALTVALUE.getBytes(),
65536, 256);
        SecretKey tmp = factory.generateSecret(spec);
        SecretKeySpec secretKey = new SecretKeySpec(tmp.getEncoded(), "AES");
        Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
        cipher.init(Cipher.ENCRYPT_MODE, secretKey, ivspec);
        /* Retruns encrypted value. */
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
           return Base64.getEncoder()
                 .encodeToString(cipher.doFinal(strToEncrypt.getBytes(StandardCharsets.UTF_8)));
        }
      }
      catch (InvalidAlgorithmParameterException | InvalidKeyException | NoSuchAlgorithmException |
InvalidKeySpecException | BadPaddingException | IllegalBlockSizeException |
NoSuchPaddingException e)
      {
        System.out.println("Error occured during encryption: " + e.toString());
      }
      return null;
   }
//-------------------------------------------------------------------------
//-------------------------------------------------------------------------

   /* Decryption Method */
   public static String decrypt(String strToDecrypt)
   {
      try
      {
        /* Declare a byte array. */
```

```
            byte[] iv = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
            IvParameterSpec ivspec = new IvParameterSpec(iv);
            /* Create factory for secret keys. */
            SecretKeyFactory factory = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256");
            /* PBEKeySpec class implements KeySpec interface. */
            KeySpec spec = new PBEKeySpec(SECRET_KEY.toCharArray(), SALTVALUE.getBytes(),
65536, 256);
            SecretKey tmp = factory.generateSecret(spec);
            SecretKeySpec secretKey = new SecretKeySpec(tmp.getEncoded(), "AES");
            Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5PADDING");
            cipher.init(Cipher.DECRYPT_MODE, secretKey, ivspec);
            /* Retruns decrypted value. */
            if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
                return new String(cipher.doFinal(Base64.getDecoder().decode(strToDecrypt)));
            }
        }
        catch (InvalidAlgorithmParameterException | InvalidKeyException | NoSuchAlgorithmException |
InvalidKeySpecException | BadPaddingException | IllegalBlockSizeException |
NoSuchPaddingException e)
        {
            System.out.println("Error occured during decryption: " + e.toString());
        }
        return null;


    //----------------------------------------------------------------------
// Java implementation for Custom
// Encryption and Decryption of String
        // Function to encrypt the String
    public static String SimpleEncryption(String str) {
```

```
        int n=str.length();
        char c[] =str.toCharArray();
        for(int i=0;i<n;i++)
            c[i]=(char)(c[i]+2);

        // System.out.println(c);
        String string = new String(c);
        // System.out.println(string);
        return string;
        //return c1.toString();
    }
    public static String SimpleDecryption(String str) {
        int n=str.length();
        char c[] =str.toCharArray();
        for(int i=0;i<n;i++)
            c[i]=(char)(c[i]-2);

        // System.out.println(c);
        String string = new String(c);
        // System.out.println(string);
        return string;
        //return c1.toString();
    }
}
```

3) ChatListAdaptor :

```
package com.example.section7findfriend.chat;

import android.content.Context;
```

```
import android.content.Intent;
import android.util.Log;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.ImageView;
import android.widget.LinearLayout;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import com.example.section7findfriend.R;

import java.util.List;

public class ChatListAdaptor extends RecyclerView.Adapter<ChatListAdaptor.ChatListViewHolder> {

    //Local variable for Adaptor
    private Context context;
    private List<ChatListModel> chatListModelList;

    public ChatListAdaptor(Context context, List<ChatListModel> chatListModelList) {
        this.context = context;
        this.chatListModelList = chatListModelList;
    }
    //*************************************************************
```

```
    @NonNull
    @Override
    public ChatListAdaptor.ChatListViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int
viewType) {
        View view = LayoutInflater.from(context).inflate(R.layout.chat_list_layout,parent,false);

        return new ChatListViewHolder(view);
    }
    @Override
    public void onBindViewHolder(@NonNull ChatListAdaptor.ChatListViewHolder holder, int
position) {

        ChatListModel chatListModel = chatListModelList.get(position);

        holder.fullName.setText(chatListModel.getUserName());

        //Done.....

        holder.cl_ll.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                Intent intent = new Intent(context, ChatActivity.class);
                intent.putExtra("user_key",chatListModel.getUserId());
                intent.putExtra("user_Name",chatListModel.getUserName());
                Log.d("","User_key : Before Pass "+chatListModel.getUserId());
                context.startActivity(intent);
            }
        });
```

```
    }

    @Override
    public int getItemCount() {
        return chatListModelList.size();
    }
//*************************************************************************************
    public class ChatListViewHolder  extends  RecyclerView.ViewHolder{

        private LinearLayout linearLayout;
        private TextView fullName;
        private TextView lastMessage;
        private TextView lastMessageTime;
        private TextView unreadCount ;
        private ImageView clProfile;
        private LinearLayout cl_ll;
        public ChatListViewHolder(@NonNull View itemView) {
            super(itemView);
            linearLayout = itemView.findViewById(R.id.cl_ll);
            fullName=itemView.findViewById(R.id.cl_fullName);
            lastMessage=itemView.findViewById(R.id.cl_lastMsg);
            lastMessageTime=itemView.findViewById(R.id.cl_lastMsgTime);
            unreadCount=itemView.findViewById(R.id.cl_UnreadCount);
            clProfile=itemView.findViewById(R.id.cl_Profile);
            cl_ll=itemView.findViewById(R.id.cl_ll);
        }
    }
}
```

4) chatListModel :

Activity_chat.xml

```
package com.example.section7findfriend.chat;


public class ChatListModel {
    private  String userId;
    private String userName;
    private String unreadCount;
    private  String lastMessage;
    private  String lastMessageTime;



    public ChatListModel(String userId, String userName, String unreadCount, String lastMessage, String
lastMessageTime) {
        this.userId = userId;
        this.userName = userName;
        this.unreadCount = unreadCount;
        this.lastMessage = lastMessage;
        this.lastMessageTime = lastMessageTime;
    }



    public String getUserId() {
        return userId;
    }

    public void setUserId(String userId) {
        this.userId = userId;
```

```
}

public String getUserName() {
    return userName;
}

public void setUserName(String userName) {
    this.userName = userName;
}

public String getUnreadCount() {
    return unreadCount;
}

public void setUnreadCount(String unreadCount) {
    this.unreadCount = unreadCount;
}

public String getLastMessage() {
    return lastMessage;
}

public void setLastMessage(String lastMessage) {
    this.lastMessage = lastMessage;
}

public String getLastMessageTime() {
    return lastMessageTime;
```

```
    }

    public void setLastMessageTime(String lastMessageTime) {
        this.lastMessageTime = lastMessageTime;
    }
```

Frament_chat-xml

```
\<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@drawable/bgtry1hp"
    tools:context=".chat.ChatFragment" >

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/rvChats"
        android:layout_width="0dp"
        android:layout_height="0dp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.0"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.0" />
```

```
    <TextView
        android:id="@+id/tvEmptyChat"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginStart="8dp"
        android:layout_marginTop="8dp"
        android:layout_marginEnd="8dp"
        android:layout_marginBottom="8dp"
        android:text="@string/empty_chat_list"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Activity_splash.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".Splash.SplashActivity"
    android:background="@drawable/try1">

    <ImageView
        android:id="@+id/ivSplash"
        android:layout_width="276dp"
        android:layout_height="267dp"
        android:layout_marginBottom="90dp"
```

```
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.496"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.55"
        app:srcCompat="@drawable/logo" />

    <TextView
        android:id="@+id/tvSplash"
        android:layout_width="396dp"
        android:layout_height="156dp"
        android:fontFamily="@font/josefinsansbolditalic"
        android:gravity="center"
        android:text="Secure messenger"
        android:textAlignment="center"
        android:textAllCaps="true"
        android:textColor="#191970"
        android:textSize="50dp"
        android:textStyle="italic"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.0"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.888" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Chatlist.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="75dp"
    android:layout_marginTop="5dp">

    <LinearLayout
        android:id="@+id/cl_ll"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
```

```
app:layout_constraintEnd_toEndOf="parent"
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintTop_toTopOf="parent">

<ImageView
   android:id="@+id/cl_Profile"
   android:layout_width="70dp"
   android:layout_height="70dp"
   android:layout_gravity="clip_vertical"
   app:srcCompat="@drawable/ic_person_logo" />

<LinearLayout
   android:layout_width="wrap_content"
   android:layout_height="match_parent"
   android:layout_marginLeft="10dp"
   android:layout_weight="1"
   android:orientation="vertical">

   <TextView
      android:id="@+id/cl_fullName"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:maxLines="1"
      android:text="UserName"
      android:textColor="#000000"
      android:textSize="25dp" />

   <TextView
      android:id="@+id/cl_lastMsg"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:layout_marginTop="6dp"
      android:maxLines="1"
      android:text="Last Message" />
</LinearLayout>

<LinearLayout
   android:layout_width="wrap_content"
   android:layout_height="wrap_content"
   android:layout_gravity="right|center_vertical"
   android:orientation="vertical">

   <TextView
      android:id="@+id/cl_UnreadCount"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
```

```
            android:layout_gravity="right"
            android:layout_marginRight="10dp"
            android:background="@drawable/red_circle"
            android:gravity="center"
            android:padding="8dp"
            android:text="3"
            android:textColor="#FFFFFF"
            android:textSize="20dp" />

        <TextView
            android:id="@+id/cl_lastMsgTime"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Message" />
      </LinearLayout>
   </LinearLayout>

   <View
      android:layout_width="match_parent"
      android:layout_height="0.1dp"
      android:layout_marginTop="8dp"
      android:background="@color/gray"
      app:layout_constraintBottom_toBottomOf="parent"
      app:layout_constraintEnd_toEndOf="parent"
      app:layout_constraintStart_toStartOf="parent"
      app:layout_constraintTop_toBottomOf="@+id/cl_ll" />


</androidx.constraintlayout.widget.ConstraintLayout>
```

CONCLUSION

Secure Messenger is now complete. With the help of the Android SDK and a strong passion to create a secure and reliable chat application, we have launched our creation into the world. We have created a safe and secure platform for friends, family, and business associates to communicate in real-time without having to worry about the security of their conversations. With encryption, decryption, data transmission, registration, and login protocols, Secure Messenger puts privacy first. And since the app is open source, users can rest assured that their data will remain secure. We hope that Secure Messenger will revolutionize the way people communicate and foster relationships based on trust.

FUTURE SCOPE

Future work on a secure messaging app could focus on improving front end design, security protocols, and performance improvements, as well as exploring new features and adaptions.

I). Security: Incorporating more advanced encryption protocols, adjusting server settings, and integrating two-factor authentication.

II). Performance: Introducing architecture such as multi-server authentication and segmenting the application into smaller components.

III). Features/Adaptations: Exploring features such as group messaging, voice/video conference options, and integration with other applications. Investigating adaptions such as local caching, integration with existing distribution channels, and exploring other markets and regions.

GLOSSARY

- AES: A type of symmetric-key encryption that is commonly used to secure data in transmission.
- HMAC: A type of authentication code used to verify the integrity of data.
- SSL: A Protocol for secure communication between two computers, typically over the internet.
- Diffie-Hellman: An algorithm for establishing a shared secret key between two parties over an unsecured connection.
- Public Key Cryptography: A type of cryptography that uses two different keys to encrypt and decrypt messages.
- Symmetric Key Encryption: A type of encryption that uses a single shared key to both encrypt and decrypt data.

Annexure A: Form A

MIT School of Engineering

Department of Computer Science and Engineering

Mini Project (I, II, III, IV) Topic Approval

Date: 17/11/2024

Class: TY IT

Project Group ID: 17

Group Members:

Enrollment No	Name of Student	E-mail Address	Contact
MITU21BTIT0042	**Omkar Rasal**	omkar15072003@gmail.com	9850060742
MITU21BTIT0061	**Tanvi Bodke**	tanvibodke0@gmail.com	7262066876
MITU21BTIT0005	**Ajinkya Sathe**	ajinkyasathe456@gmail.com	9420745845
MITU21BTIT0048	**Prathmesh Panchmire**	prathameshpanchmire96@gmail.com	8080011908

Project Title Evaluation Parameters:

Sr no.	Parameters	Topic 1	Topic 2	Topic 3
1.	**Title**	Secure Messenger		
2.	**Domain Expertise**	Cryptography & Security		
3.	**Technical Feasibility**	Java, Android, Firebase, API – Dexter, Glide, Lottie		
4.	**Future Scope**	Video Calls, Conference Calls		
5.	**Applicability**	Communication Platform		
	Approved (✓)	**(✓)**		
Remark:				

Sr. No.Name of Subject Expert **Signature**

1. _____ _____
2. _____ _____

Annexure B: Form B

MIT School of Engineering

Department of Computer Science and Engineering

Mini Project (I, II, III, IV,V) Topic Approval

Date: 17/11/2024

Class: TY IT

Project Group ID: Group 17

Project Title: Secure Messenger

Project Title Evaluation Parameters:

Sr. No.	Parameters	Description about Project	Marks (5)
1	Business Ideas and Implementation from project	Encrypted Communication based platform	
2	Market Survey (competitors, substitute products, potential market, etc.)	WhatsApp, Signal	
3	Market Acceptability of Product	It will be acceptable widely as it is necessary and none of the app contains all the necessary features.	
4	Emerging Trends about Project and Product	Read receipts, Profile picture, Message count of unread messages, Voice call and video Call, Online - offline status	
5	Income Generation ideas through Project	Subscription based	
6	Project Profitability	Availability of emerging features	
7	Cost Benefit Analysis	Direct costs such as the server cost, might potentially be the only cost which we will be incurring.	
8	Any Other Point	We will be adding some of encryption algorithms	
Remark:			

Commercial Feasibility of project is evaluated based on the above parameters.

Project Approval Status: Approved

(Name & Designation of Market Expert)
Signature with Date.

9. REFERENCES

1. Liu, Shuai, and Julita Vassileva. "Group Key Management for Fast Distribution of Group Cryptography Parameters in Mobile IP/Ad Hoc Group Communication." Proceedings of the International Workshop on Groupware, 1365-71, 2003.

2. Panayiotou, Alexander, and Satoshi Nakamura. "Exploring User-Friendly and High-Performance Cryptographic Actions Complementary to Face-to-Face Chatting." The Digital Journal on Advances in Security and Privacy, 9(1), 2020.

3. Hu, Hanxiao, and Ze-Xuan Tan. "E-Chat: A Mobile Secure Instant Messaging Application Based on Group chat Cryptography with Timestamp Provisions." Journal of Parallel and Distributed Computing, 67(11), 2007.

4. Dhawan, Hemanshu, and Abhishek Tiwari. "Secure End-to-End Communication System for Instant Messaging Using Advanced Cryptography." International Journal of Computer Applications, 216(4), 2016.

5. El-Raey, M.S., and M.H. Omar. "Secure Multi Key Cryptography for Secure Group Chatting." Proceedings of International Conference on University Research and Development, 4, 2013.

6. Dhawan, Abhishek, and Hemanshu Dhawan. "A Group Cryptographic System With an Aim of Establishing Key Agreement Between Chatting Parties." International Journal of Advanced Computer Research, 8(27), 2018.